EDA

Event-Driven & Asynchronous Architecture

Foundations · Messaging vs Streaming · Enterprise Kafka

Rajesh Pasham

The Problem with Synchronous Systems

Why traditional architectures break at scale

Tight Coupling

Services must know and depend on each other. A change to one ripples across the entire system.

Cascading Failures

If Service B is slow, Service A waits. Latency stacks up. One outage brings down the chain.

Bottlenecks at Scale

Synchronous calls block threads. As traffic grows, performance degrades and costs spiral.

What Is Event-Driven Architecture?

Events as first-class citizens of your system

"An event is an immutable record
of something that happened.
Not a command.
Not a request.
A fact."

Decouple

Producers publish events without knowing who listens. Consumers subscribe without knowing who produces.

React

Services respond to events asynchronously, at their own pace — no blocking, no waiting.

Scale

Add consumers, fan out streams, replay history. Grow without re-engineering the core.

Synchronous vs Asynchronous: Head-to-Head

Comparing latency, coupling, and resilience

Dimension	Synchronous	Asynchronous / Event-Driven
Latency	Cumulative — adds up per hop	Parallel — bounded by slowest consumer
Coupling	Tight — direct service calls	Loose — via shared event channel
Failure Impact	Cascades across call chain	Isolated — consumers fail independently
Scalability	Limited by bottleneck service	Horizontal — add consumers freely
Auditability	Ephemeral — no built-in log	Durable — events are the record

Five Principles of Event-Driven Design

The foundations every practitioner must know

01

Loose Coupling

Services communicate via events, never direct calls.

02

Async by Default

Producers fire and forget. Consumers react at will.

03

Immutability

Events are facts — never altered after publication.

04

Retention & Replay

Store events durably; replay to recover or onboard.

05

Elastic Scalability

Scale consumers independently to match load.

Messaging vs Event Streaming

Two patterns — different jobs, different strengths

MESSAGING

- Messages consumed once, then deleted
- Point-to-point or request–response
- Dynamic consumer pools for tasks
- No built-in replay or history
- **Best for: job queues, notifications, commands**

VS

EVENT STREAMING

- Events retained for configurable period
- Publish–subscribe, fan-out to many consumers
- Multi-consumer groups read independently
- Replay and reprocessing built in
- **Best for: analytics, fraud detection, integration**

Apache Kafka: The Enterprise Backbone

Streaming + messaging in one durable, scalable platform

Topics & Partitions

Events grouped into topics, split into partitions for parallel processing and horizontal scale.

Producers

Any system can publish events to a topic — appended to the immutable log instantly.

Consumer Groups

Each group reads the stream independently at its own offset. Scale by adding instances.

Retention & Replay

Events live on disk for days or weeks. Re-read from any offset — no impact on producers.

Why Kafka Wins in the Enterprise

Durability, ecosystem, and economics at scale

1M+

Events/sec per commodity cluster

99.9%

Durability via multi-broker replication

∞

Consumer groups reading same stream

Open Source Economics

No per-message licensing. Disk-based storage is far cheaper than RAM-based alternatives.

Rich Ecosystem

Kafka Connect for 200+ integrations. Kafka Streams for in-process processing. Flink for heavy analytics.

Operational Maturity

Battle-tested at LinkedIn, Netflix, Uber at billions of events/day. Managed offerings from all major clouds.

Real-World Enterprise Use Cases

From fraud detection to supply chain intelligence

FINANCIAL SERVICES

Real-Time Fraud Detection

< 200ms detection

RETAIL & E-COMMERCE

Omnichannel Inventory Sync

Zero stockout lag

RIDE-SHARING

Driver–Rider Matching at Scale

< 1s end-to-end

IOT & SUPPLY CHAIN

Sensor Anomaly Detection

Predictive maintenance

MARKETING TECH

Real-Time Personalization

Live profile updates

The Async Advantage: Performance Unlocked

Replacing sequential chains with parallel event processing

Synchronous Latency Math

Total = A + B + C + D
= Slowest × Number of hops

Asynchronous Latency Math

Total = max(A, B, C, D)
= Parallel execution, bounded by one

Thread Efficiency

No threads blocked waiting. CPU handles real work.

Fault Isolation

One consumer fails — others continue uninterrupted.

Independent Scaling

Hot consumers scale solo. No need to scale the world.

Taming Complexity: Patterns That Work

Eventual consistency, backpressure & failure handling

Eventual Consistency

Distributed services converge to consistent state over time. Embrace it — design idempotent consumers, use compensating transactions, and monitor convergence.

Backpressure Management

Kafka partitions act as durable buffers. When consumers lag, scale them out horizontally. Use dead-letter queues (DLQs) to isolate poison messages.

At-Least-Once Delivery

Events may be delivered more than once during retries. Build idempotent consumers that safely handle duplicates — use event IDs for deduplication.

Key Takeaways

What to carry from this session

1. Events decouple systems — produce facts, not commands, and let consumers decide what to do.
2. Async > Sync at scale — parallel execution slashes latency and isolates failures naturally.
3. Messaging handles tasks; streaming handles facts — choose based on lifecycle, not familiarity.
4. Kafka is the enterprise standard — durable, horizontally scalable, and ecosystem-rich.
5. EDA is a strategic posture — it enables organizational speed and long-term data reuse.